

CS310 Natural Language Processing
Assignment 5: DPO for LLM Human Alignment
Total points: 30

Goal: Run direct preference optimization (DPO) for a SFT version of GPT-2 model (gpt2-medium355M-sft.pth) on a simple human alignment dataset (instruction-data-with-preference.json, ~1k samples).

Submit:

- The complete training script `run_dpo.py`.
- The generated model outputs, `model_outputs.json`, whose format should follow the provided `reference_outputs.json`.
- Write up a PDF report to include the results for Requirement. (see highlights)

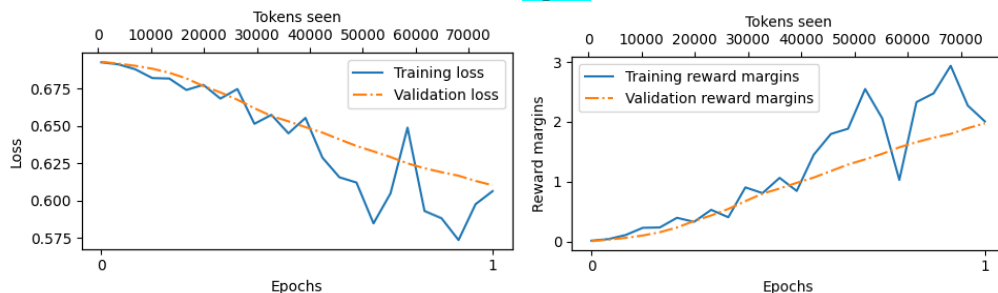
Requirements:

1) (15 points) Complete the code for running the DPO task.

- Reuse the code from the lab, including:
 - The `PreferenceDataset` class and the `custom_collate_fn` function.
 - The three functions for computing DPO loss: `compute_dpo_loss`, `compute_logprobs`, and `compute_dpo_loss_batch`.
 - A new `train_model_dpo_simple` function as the main training loop. It takes the two models π_θ (`policy_model`) and π_{ref} (`reference_model`) as inputs, along with other arguments.
`policy_model` should be under training mode by calling `.train()` (by default); `reference_model` should be under evaluation model by calling `.eval()` (no updates).
 - Other than the training and validation losses, the *reward margins* should also be tracked:
 $\text{chosen_reward} := \log \frac{\pi_\theta(y^+|x)}{\pi_{\text{ref}}(y^+|x)}$, $\text{rejected_reward} := \log \frac{\pi_\theta(y^-|x)}{\pi_{\text{ref}}(y^-|x)}$
 $\text{reward_margin} := \text{chosen_reward} - \text{rejected_reward}$

2) (8 points) Run DPO training, save the output loss and reward margin curves, and the resulting model.

- Training of 1 epoch takes about 2~3 minutes on a A100 GPU.
- You expect to see the losses dropping while the reward margins increasing (see examples below). **Include the curve plots in your report.**



3) (7 points) Generate responses on the AlpacaEval dataset and then run the evaluation process.

- Install the AlpacaEval package following the instructions in the repo:
https://github.com/tatsu-lab/alpaca_eval
- The evaluation can be done by running the following command:

```
$ OPENAI_CLIENT_CONFIG_PATH='/path/to/your/openai_configs.yaml'
alpaca_eval evaluate \
  --model_outputs '/path/to/your/model_outputs.json' \
  --reference_outputs '/path/to/the/reference_outputs.json' \
  --annotators_config '/full/path/to/the/folder/qwen_judge'
```

- Note that the `--annotators_config` only accepts full path to `qwen_judge`. (This folder's content files are provided in `qwen_judge.zip`)
- You should manually generate the `model_outputs.json` using your DPO trained model. Its format should be the same as the provided `alpaca_eval.json` and `reference_outputs.json`.
The generation can be done using a provided script `generate_dpo_response.py`.
- You might have network issues running the script if you are using `hf-mirror.com`. If that happens, add an environment variable `"HF_ENDPOINT=https://huggingface.co"`.

- When the evaluation is done, the results will be saved to a `leaderboard.csv` and `annotations.json` files. A summary will also be printed in the command line:

	length_controlled_winrate	win_rate	standard_error	n_total	avg_length
example	33.19	15.79	1.15	805	327

Include a screenshot of this win rate results in your report.

You should expect a win rate $>50\%$, as the reference outputs are generated using a non-aligned GPT2-sft model.

- Refer to `alpacaeval_guideline.md` for a full tutorial.